

TECH

REACT CORE

RENDERING **HOOKS** EFFECTS REACT 18+ / CONCURRENCY

HOOKS

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК react.dev

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ЛЁГКИЙ	ПРАВИЛА ХУКОВ Q1 · Q2	УСЛОВНЫЕ ВЫЗОВЫ	Базовые правила, без которых хуки ломаются.
ДЕНЬ 2 СРЕДНИЙ	useState Q3 · Q4 · Q5	useReducer	Когда какой выбрать.
ДЕНЬ 3 СРЕДНИЙ	useRef Q6 · Q7	REF vs STATE	Когда нужен мутабельный контейнер вместо state.
ДЕНЬ 4 СРЕДНИЙ	useMemo ЗАЧЕМ Q8 · Q10 · Q12	useCallback	React.memo — Три инструмента мемоизации в нормальном виде.
ДЕНЬ 5 СРЕДНИЙ	КОГДА МЕМО ВРЕДЕН ДЕФОЛТ Q9 · Q11 · Q13	БЕСПОЛЕЗЕН	НЕ Контр-аргументы — когда мемоизация лишняя.
ДЕНЬ 6 ТЯЖЁЛЫЙ	STALE CLOSURE Q14 · Q15 · Q16	FUNCTIONAL UPDATE	Главная боль хуков и три способа лечения.
ДЕНЬ 7 СРЕДНИЙ	CUSTOM HOOKS Q17 · Q18 · Q19 · Q20	ТЕСТ	HIDDEN GLOBAL Дизайн собственных хуков и где не превратить их в скрытое глобальное состояние.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ЛЁГКИЙ 2 ВОПРОСА

ПРАВИЛА ХУКОВ УСЛОВНЫЕ ВЫЗОВЫ

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Базовые правила, без которых хуки ломаются.

ВОПРОСЫ ДНЯ

Q01 Какие правила hooks?

Q02 Почему hooks нельзя вызывать условно?

Какие правила hooks?

ОПИСАНИЕ

Два главных правила: вызывать хуки только на верхнем уровне функции (не в условиях, циклах, вложенных функциях) и только из React-компонентов или других хуков. Это нужно, чтобы React мог сопоставить вызовы хуков с их state по порядку – внутренняя реализация опирается на стабильную последовательность.

КАК РАБОТАЕТ

- 01 Top-level: хук вызывается до любых return/if.
- 02 Только из компонентов или custom-hooks (имя use*).
- 03 ESLint-плагин react-hooks/rules-of-hooks ловит нарушения.
- 04 React внутренне нумерует хуки по порядку вызова и связывает с storage в Fiber-узле.
- 05 Любое условное пропускание хука сдвигает нумерацию – state «утечёт» к другому хуку.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой компонент с состоянием/effect – обязательное правило.
- 02 Custom-хуки – те же правила.
- 03 Линтер в CI – must-have для команды.

ПОДВОДНЫЕ КАМНИ

- 01 if (cond) useState() – баг, при смене cond state съезжает.
- 02 Ранний return перед useEffect – то же самое.
- 03 useEffect внутри map/forEach – нарушение.
- 04 Хук в обычной функции (не use*) – нет связи с компонентом.

КОД

// Плохо

```
function C({ flag }) {  
  if (flag) useState(0); // ← нельзя  
  return null;  
}
```

// Хорошо

```
function C({ flag }) {  
  const [n, setN] = useState(0);  
  if (!flag) return null;  
  return <div>{n}</div>;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт без линтера?
- 02 Можно ли хуки в callback?
- 03 Зачем имя начинается с use?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/rules/rules-of-hooks>

Почему hooks нельзя вызывать условно?

ОПИСАНИЕ

React связывает каждый `useState/useEffect/etc` с конкретной ячейкой во внутреннем `list`-е по ПОРЯДКУ вызова. Если на одном рендере хук вызывается, на другом нет – все следующие хуки сместятся, и значения попадут не в свои ячейки. Это ломает `state`, `effect-deps`, `ref`.

КАК РАБОТАЕТ

- 01 Хуки не используют имя – только `index` в порядке вызова.
- 02 Условный пропуск = смещение всей последовательности.
- 03 В dev React выкидывает ошибку «`Rendered fewer/more hooks than during the previous render`».
- 04 Альтернатива: всегда вызывать хук, условно использовать результат.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая динамическая логика – внутри хука, не вокруг него.
- 02 Условный рендер компонента – раньше `return`, после хуков.

ПОДВОДНЫЕ КАМНИ

- 01 Loops: хук в `for`-цикле – нельзя (порядок зависит от длины).
- 02 Switch: хук в `case` – нельзя.
- 03 `try/catch` вокруг хука – обычно ок, если попадает на каждый рендер.

КОД

// Условное использование результата

```
function C({ flag }) {  
  const [n, setN] = useState(0);  
  const value = flag ? n : 0; // условие – на результат  
  return <div>{value}</div>;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как React хранит `state` хука?
- 02 Что покажет `dev-error` при нарушении?
- 03 Можно ли разделить компонент, чтобы условно использовать хук?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/rules/rules-of-hooks>

ДЕНЬ 2

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

useState useReducer

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Когда какой выбрать.

ВОПРОСЫ ДНЯ

Q03 Когда нужен useState?

Q04 Когда нужен useReducer?

Q05 Когда useReducer лучше useState?

Когда нужен useState?

ОПИСАНИЕ

useState — для локального состояния компонента, простого по структуре (примитив, плоский объект, массив). Это стандартная единица state в React. Если update-логика сложная (зависит от предыдущего state, есть несколько связанных значений) — useReducer обычно чище.

КАК РАБОТАЕТ

- 01 [state, setState] = useState(initial).
- 02 setState(value) — новая ссылка → rerender.
- 03 setState(prev => next) — функциональный апдейт.
- 04 Lazy initial: useState(() => heavyCompute()) — вычисляется один раз.
- 05 Object.is-сравнение — тот же state не вызывает rerender.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Простые поля: input value, isOpen, count.
- 02 Локальный UI-state.
- 03 Toggle, счётчики, фильтры.

ПОДВОДНЫЕ КАМНИ

- 01 Хранить ВСЁ в одном объекте useState — теряется тонкая оптимизация рендера.
- 02 Мутация state без новой ссылки → rerender пропускается.
- 03 Тяжёлая инициализация без lazy — выполняется на каждом рендере.

КОД

```
// Lazy init
const [list, setList] = useState(() => loadHeavy());
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда useReducer лучше?
- 02 Что вернёт setState(state) с тем же значением?
- 03 Зачем lazy init?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState>

ВОПРОС Q04

Когда нужен `useReducer`?

ОПИСАНИЕ

`useReducer` – для state со сложной update-логикой, несколькими связанными полями или нетривиальными переходами. Reducer-функция превращает (state, action) в next state. Дисциплинирует астейты, упрощает тестирование, естественно ложится на discriminated union actions и state machines.

КАК РАБОТАЕТ

- 01 `[state, dispatch] = useReducer(reducer, initial).`
- 02 `reducer(state, action) → next state.`
- 03 Lazy init: `useReducer(reducer, arg, init).`
- 04 Действия – обычно DU: `{ type: 'add', payload }`.
- 05 В сочетании с TypeScript – exhaustive check по `action.type`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Многопольный state с зависимостями (form-state).
- 02 State machines (idle/loading/success/error).
- 03 Множественные действия от пользователя.
- 04 Когда хочется отделить «как меняется» от «что меняется».

ПОДВОДНЫЕ КАМНИ

- 01 Reducer должен быть pure – никаких fetch внутри.
- 02 Разрастающийся reducer – стоит делить.
- 03 Side effects – отдельно (effect/middleware).
- 04 Большой initial state – lazy init.

КОД

// Reducer

```
type Action = { type: 'inc' } | { type: 'set'; value: number };
function r(state: number, a: Action) {
  switch (a.type) {
    case 'inc': return state + 1;
    case 'set': return a.value;
  }
}
const [n, dispatch] = useReducer(r, 0);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем reducer хорош для тестов?

- 02 Как сочетать с context для глобального state?
- 03 Когда лучше Zustand/Redux вместо useReducer?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useReducer>

ВОПРОС Q05

Когда useReducer лучше useState?

ОПИСАНИЕ

Когда state стал сложным или трансформируется по правилам, плохо помещающимся в один setState. Признаки: 5+ полей, зависящих друг от друга; действия описывают семантику (не «set field», а «начать загрузку»); одинаковые update-paths повторяются. Reducer централизует логику, делает компонент тоньше.

КАК РАБОТАЕТ

- 01 Action-types отражают семантику, не «как обновить поле».
- 02 Reducer тестируется без React – просто чистая функция.
- 03 DU actions + DU state – exhaustive check сразу.
- 04 Dispatch стабилен – не нужно мемоизировать как callback.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form-state с зависимостями.
- 02 State machines (wizard, плеер).
- 03 Многошаговые workflow.
- 04 Когда хочется логировать/replay actions.

ПОДВОДНЫЕ КАМНИ

- 01 Не каждый useState нужно превращать в reducer – лишняя сложность.
- 02 Reducer не отменяет необходимость effects.
- 03 Большой reducer – кандидат на разбиение.

КОД

```
// Сложный state – reducer
// state = idle | loading | success | error
// action = fetch | succeed | fail
// reducer описывает все переходы – компонент только
// читает state.kind
```

МОГУТ ДОСПРОСИТЬ

- 01 Что чище – useReducer или несколько useState?
- 02 Как тестировать reducer?
- 03 Когда reducer становится «слишком большим»?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/extracting-state-logic-into-a-reducer>

useRef REF vs STATE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Когда нужен мутабельный контейнер вместо state.

ВОПРОСЫ ДНЯ

Q06 Когда нужен useRef?

Q07 Чем изменение ref отличается от state update?

ВОПРОС Q06

Когда нужен useRef?

ОПИСАНИЕ

useRef нужен для двух задач: ссылка на DOM-узел (`<input ref={ref} />`) и хранение мутабельного значения между рендерами без триггера `rerender`. Ref сохраняется одним и тем же объектом на все рендеры; меняется `.current` – `rerender` НЕ вызывается.

КАК РАБОТАЕТ

- 01 `useRef(initial) → { current: initial }`.
- 02 Объект стабилен на все рендеры.
- 03 Запись `.current` не вызывает `rerender`.
- 04 Чтение `.current` безопасно в `effect`, обработчиках, теле после `mount`.
- 05 В `render`-фазе чтение допустимо, запись – антипаттерн.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DOM-доступ: `focus`, `scroll`, `getBoundingClientRect`.
- 02 Хранить «свежее» значение для замыкания (см. Q14).
- 03 Сохранять `id` таймера / подписки между рендерами.
- 04 `Counter`, не влияющий на UI (рендер-метрика).

ПОДВОДНЫЕ КАМНИ

- 01 Использовать `ref` как скрытый `state` – отказ от реактивности.
- 02 Запись `ref` в `render` – ломает `pure-render`.
- 03 `ref` на компонент без `forwardRef` – не работает.
- 04 Reading `ref` сразу после `render` – `undefined` (DOM-узел появится в `commit`).

КОД

```
// DOM-доступ
const ref = useRef<HTMLInputElement>(null);
useEffect(() => { ref.current?.focus(); }, []);

```

```
// Свежее значение для callback
const latest = useRef(value);
useEffect(() => { latest.current = value; });
// в долгоживущем колбэке: latest.current
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `ref` отличается от `state`?
- 02 Что такое `forwardRef`?

03 Когда ref недоступен после render?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useRef>

ВОПРОС Q07

Чем изменение ref отличается от state update?

ОПИСАНИЕ

State update создаёт новую ссылку и триггерит rerender; ref-обновление – мутация одного и того же объекта без rerender. State – реактивный, виден в JSX. Ref – невидимый, его смена не отражается в UI до следующего rerender (вызванного чем-то другим).

КАК РАБОТАЕТ

- 01 `setState(v)` → новая ссылка → rerender.
- 02 `ref.current = v` → мутация → НЕТ rerender.
- 03 Ref-объект один и тот же между рендерами.
- 04 В JSX `ref.current` «замораживается» в момент рендера – следующее изменение не отрисуется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 State – для всего, что должно быть в UI.
- 02 Ref – для хранения, которое не должно вызывать rerender.
- 03 Ref для свежего значения долгоживущего колбэка.
- 04 Ref для DOM-доступа.

ПОДВОДНЫЕ КАМНИ

- 01 Использовать ref для значения, которое нужно в UI – UI не обновится.
- 02 Использовать state для вещи, которая не должна вызывать rerender – лишняя работа.
- 03 Мутировать состояние внутри ref-объекта вместо state – трудно дебажить.

КОД

// Контраст

```
const [n, setN] = useState(0);
const renderCount = useRef(0);
renderCount.current++;
// renderCount меняется без rerender
// n меняется только при setN – и rerender случится
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда ref видно в UI?
- 02 Можно ли использовать ref как замену state?
- 03 Что произойдёт, если в JSX отрендерить `ref.current`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useRef>

ДЕНЬ 4

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

useMemo**useCallback****React.memo – ЗАЧЕМ****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три инструмента мемоизации в нормальном виде.

ВОПРОСЫ ДНЯ

Q08 Когда нужен useMemo?

Q10 Когда нужен useCallback?

Q12 Когда нужен React.memo?

Когда нужен useМемо?

ОПИСАНИЕ

useМемо нужен в двух случаях: дорогое вычисление, которое не должно повторяться без причины (фильтрация большого списка, тяжёлая трансформация); стабилизация ссылки на значение, которое потом передаётся в React.мемо-обёрнутый компонент или в deps другого хука. Без таких причин useМемо обычно лишний.

КАК РАБОТАЕТ

- 01 useМемо(() => compute(deps), [deps]) – кеширует результат до смены deps.
- 02 Возвращает ту же ссылку, пока deps не поменяются (по Object.is).
- 03 В Strict Mode dev может выполнить дважды.
- 04 С React Compiler во многих случаях не нужен.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлая фильтрация/сортировка большого массива.
- 02 Создание объекта-конфига, передаваемого в мемо-компонент.
- 03 Стабилизация deps для useEffect.

ПОДВОДНЫЕ КАМНИ

- 01 Дешёвое вычисление + useМемо = чистый overhead.
- 02 Inline-функция как фабрика – useCallback логичнее.
- 03 Неправильные deps → возвращает устаревший результат.

КОД

// Дорогая фильтрация

```
const visible = useМемо(
  () => items.filter(x => x.tag === tag),
  [items, tag]
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем useМемо отличается от useCallback?
- 02 Что делает React Compiler с useМемо?
- 03 Что произойдёт без useМемо на тяжёлом фильтре?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useМемо>

ВОПРОС Q10

Когда нужен useCallback?

ОПИСАНИЕ

`useCallback` стабилизирует ссылку на функцию между рендерами. Нужен, когда функция передаётся в `React.memo`-компонент или в `deps` другого хука. В отрыве от этих сценариев – `useless`: пересоздание функции на каждом рендере дешевле, чем сравнение `deps`.

КАК РАБОТАЕТ

- 01 `useCallback(fn, deps) === useMemo(() => fn, deps)`.
- 02 Возвращает ту же ссылку, пока `deps` не поменялись.
- 03 Не оптимизирует саму функцию – оптимизирует ссылку.
- 04 С `React Compiler` автоматизируется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Колбэк в `React.memo`-компонент.
- 02 Колбэк в `deps` другого `useEffect/useMemo`.
- 03 Стабильная ссылка для `useImperativeHandle`.

ПОДВОДНЫЕ КАМНИ

- 01 `useCallback` без `React.memo` вниз по дереву – бесполезен.
- 02 Зависимые callbacks с `deps` на `state` – старая ссылка захватывает старое `state` (`stale closure`).
- 03 Чрезмерное использование делает код шумным.

КОД

// Стабильный handler

```
const onClick = useCallback(() => doSth(id), [id]);  
<MemoButton onClick={onClick} />
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `useCallback` бесполезен?
- 02 Что такое `stale closure` в callback?
- 03 Чем `useEvent` (RFC) удобнее `useCallback`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useCallback>

Когда нужен React.memo?

ОПИСАНИЕ

React.memo оборачивает компонент и пропускает render, если новые props === старым (shallow). Нужен, когда: компонент тяжёлый и props стабильны; родитель часто рендерится по причинам, не касающимся потомка; рендер потомка дорог (большое поддерево, тяжёлые вычисления).

КАК РАБОТАЕТ

- 01 React.memo(C) → новый компонент с pre-render-проверкой.
- 02 Shallow compare props через Object.is.
- 03 Custom comparator: React.memo(C, areEqual).
- 04 С children-prop часто бесполезен (новая JSX-ссылка каждый раз).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлая карточка списка.
- 02 Дорогая визуализация (canvas, SVG).
- 03 Wrapper над компонентом библиотеки.

ПОДВОДНЫЕ КАМНИ

- 01 Inline-функция/объект в props – мемо не сработает.
- 02 Children-prop – обычно ломает мемо.
- 03 Custom comparator с deep equal – медленнее наивного render.

КОД

// Базовое

```
const Heavy = React.memo(function Heavy(props) {  
  return <ExpensiveTree {...props} />;  
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда React.memo бесполезен?
- 02 Чем shallow compare отличается от deep?
- 03 Можно ли мемоизировать с children?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/memo>

ДЕНЬ 5 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

КОГДА МЕМО ВРЕДЕН БЕСПОЛЕЗЕН НЕ ДЕФОЛТ**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Контр-аргументы — когда мемоизация лишняя.

ВОПРОСЫ ДНЯ

Q09 Когда useМемо вреден?

Q11 Когда useCallback бесполезен?

Q13 Почему memoization не должна быть дефолтной реакцией на performance-проблемы?

Когда useMemo вреден?

ОПИСАНИЕ

Когда вычисление дешёвое, useMemo добавляет overhead (хранение closure, сравнение deps, ссылка) больший, чем сама работа. Также вреден, когда deps указаны неправильно (вернёт устаревший результат) и когда им пытаются «починить» баги, источник которых в другом.

КАК РАБОТАЕТ

- 01 useMemo сам стоит несколько микросекунд (closure + сравнение deps).
- 02 Для дешёвых операций затраты больше выгоды.
- 03 Неправильные deps → стейл-результат.
- 04 Не лечит баги нестабильности ссылок (нужна архитектура).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 В горячем пути render критично – мерять, не угадывать.
- 02 В тестах – проверять, что без useMemo тоже работает.

ПОДВОДНЫЕ КАМНИ

- 01 Маркер «оптимизировано» без замеров.
- 02 Слишком много useMemo замусоривает компонент.
- 03 С React Compiler ручной useMemo обычно избыточен.

КОД

```
// Лишний useMemo
// дешёвое выражение – useMemo не нужен:
const sum = useMemo(() => a + b, [a, b]);
// напрямую: const sum = a + b;
```

МОГУТ ДОСПРОСИТЬ

- 01 Как понять, нужен ли useMemo?
- 02 Чем отличается измерение в dev и prod?
- 03 Что меняет React Compiler?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useMemo>

ВОПРОС Q11

Когда useCallback бесполезен?

ОПИСАНИЕ

Когда возвращённая функция не идёт в React.мемо-компонент и не используется в deps другого хука. Например, callback для нативного DOM-обработчика, JSX без мемо-обёрнутого потомка, разовый эффект. В таком случае пересоздание функции на каждом рендере не имеет негативных последствий, а useCallback добавляет сложность.

КАК РАБОТАЕТ

- 01 Если потребитель колбэка не сравнивает ссылки — стабильность ссылки бесполезна.
- 02 DOM-обработчики (onClick={...}) и так пересоздаются на каждом рендере.
- 03 Простые компоненты без React.мемо не выигрывают от стабильной ссылки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Только если ниже по дереву есть React.мемо / useCallback / useEffect deps.
- 02 Иначе — обычная стрелочная функция в JSX.

ПОДВОДНЫЕ КАМНИ

- 01 Cargo-cult: оборачивать всё подряд.
- 02 useCallback с deps на state в долгоживущем колбэке = stale closure.
- 03 useCallback с пустыми deps + ref.current — альтернатива (см. useEvent).

КОД

// Бесполезный

```
function Btn() {
  const onClick = useCallback(() => alert('!'), []);
  return <button onClick={onClick}>X</button>;
  // ничего не выигрываем
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что выгоднее — useCallback или ref-pattern?
- 02 Когда callback в JSX дешевле обёрнутого?
- 03 Где useCallback ловит stale-closure?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useCallback>

Почему memoization не должна быть дефолтной реакцией на performance

ОПИСАНИЕ

Мемоизация лечит конкретный класс проблем – лишние ререндеры из-за нестабильных ссылок. Большинство perf-проблем – другие: тяжёлые вычисления (виртуализация / web worker), большие списки (windowing), лишние запросы (кэш / debounce), плохая архитектура context (сужение). Преждевременная мемоизация только усложняет код.

КАК РАБОТАЕТ

- 01 Сначала измерить – Profiler / Performance, найти горячую точку.
- 02 Понять класс проблемы: render, network, layout, JS.
- 03 Применить точечное лечение: virtualization, debounce, split context, lazy load.
- 04 Мемоизация – последний шаг для конкретного компонента.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Profiling-дашборды.
- 02 Code-review с акцентом на правильные перф-фиксы.
- 03 Ретроспектива после перф-инцидентов.

ПОДВОДНЫЕ КАМНИ

- 01 «Завернём всё в мемо» – много кода, мало пользы.
- 02 Игнорирование архитектурных проблем (большой context).
- 03 useМемо на серверных компонентах не имеет смысла.

КОД

// Иерархия лечений

```
// 1. virtualization (список)
// 2. context split (узкие потребители)
// 3. lazy + Suspense
// 4. мемо + useCallback (точно)
```

МОГУТ ДОСПРОСИТЬ

- 01 С чего начать оптимизацию большого приложения?
- 02 Когда виртуализация важнее мемоизации?
- 03 Как выявить «большой context»-проблему?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit>

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

3 ВОПРОСА

STALE CLOSURE**FUNCTIONAL UPDATE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Главная боль хуков и три способа лечения.

ВОПРОСЫ ДНЯ

Q14 Что такое stale closure в hook?

Q15 Как избежать stale state update?

Q16 Когда использовать functional state update?

Что такое stale closure в hook?

ОПИСАНИЕ

Stale closure – ситуация, когда колбэк в хуке захватил старые значения state/props и продолжает их использовать после ререндеров. В React это проявляется чаще всего в `useEffect` без зависимости на нужное значение, в `setInterval`-колбэке, в подписках. Лечение: правильные deps, `useRef` для свежей ссылки, или функциональный апдейт `setState(prev => ...)`.

КАК РАБОТАЕТ

- 01 Каждый рендер создаёт новые переменные state и props.
- 02 Колбэки замыкаются на ИХ значения.
- 03 Если колбэк сохранён (`useEffect` без deps, `useCallback` с пустыми deps) – он видит старое.
- 04 `useRef` + `sync-effect` – кладём свежее в `.current`.
- 05 `Functional update` – `setN(prev => ...)` не зависит от замыкания.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `useEffect` с `setInterval`/`setTimeout`/`subscribe`.
- 02 `useCallback` с зависимостями.
- 03 `addEventListener` в `useEffect` с пустыми deps.

ПОДВОДНЫЕ КАМНИ

- 01 Линтер советует добавить deps – иногда правильнее переписать на `functional update` / `ref`.
- 02 `useCallback` без правильных deps – stale.
- 03 В `Strict Mode dev` двойной mount помогает ловить.

КОД

// setInterval-бар

```
useEffect(() => {  
  const id = setInterval(() => console.log(n), 1000);  
  return () => clearInterval(id);  
}, []); // n не в deps – всегда 0
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `functional update` лечит stale?
- 02 Когда выбрать `ref`, а когда – добавить deps?
- 03 Что делает `useEvent` (RFC)?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

Как избежать stale state update?

ОПИСАНИЕ

Stale update – когда `setState` получает аргумент, вычисленный из устаревшего state. Пример: `setN(n + 1)` вызвано дважды подряд – оба видят одно и то же `n`, результат `+1` вместо `+2`. Решение – функциональная форма: `setN(prev => prev + 1)`. `prev` приходит из React и всегда свежее.

КАК РАБОТАЕТ

- 01 `setState(value)` – `value` использует state из замыкания.
- 02 `setState(prev => next)` – `prev` из React, свежее.
- 03 Несколько `setState` в одном обработчике – Functional форма гарантирует, что каждый видит результат предыдущего.
- 04 С `automatic batching` – один `render`, но цепочка применений.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Несколько `setN` в одном обработчике.
- 02 `setState` внутри `setTimeout` / `fetch.then`.
- 03 Любой `setState`, зависящий от прошлого.

ПОДВОДНЫЕ КАМНИ

- 01 Не функциональная форма – старые значения.
- 02 Объект-state – нужно `spread` с новой ссылкой: `prev => ({ ...prev, x: 1 })`.
- 03 `Reducer` уже функциональный сам по себе.

КОД

// Двойной inc

```
// плохо
setN(n + 1);
setN(n + 1); // оба используют то же n

// хорошо
setN(prev => prev + 1);
setN(prev => prev + 1); // +2 в итоге
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда обязательно `functional update`?
 - 02 Связано ли это с `automatic batching`?
 - 03 Чем `reducer` проще в этом плане?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState#updating-state-based-on-the-previous-state>

Когда использовать functional state update?

ОПИСАНИЕ

Всегда, когда новое значение зависит от старого. Это избавляет от stale-closures и от ошибок batching. Bonus: функциональный апдейт не требует state в deps useCallback/useEffect – что упрощает код.

КАК РАБОТАЕТ

- 01 `setState(prev => f(prev)).`
- 02 `prev` – гарантированно свежий.
- 03 Применяется в порядке вызовов внутри одного обработчика.
- 04 Возвращаемое значение – новый state.
- 05 `Object.is`-сравнение работает: `prev === next` → пропуск `render`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Toggles: `setOpen(p => !p).`
- 02 Counters: `setN(p => p + 1).`
- 03 Списки: `setItems(p => [...p, x]).`
- 04 Любая логика с зависимостью от прошлого.

ПОДВОДНЫЕ КАМНИ

- 01 Возврат той же ссылки – пропуск `render` (что иногда хочется, иногда нет).
- 02 Функция должна быть `pure` – без побочных эффектов.
- 03 Не путать с `useReducer dispatch`.

КОД

// Toggle

```
<button onClick={() => setOpen(p => !p)}>X</button>
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт `setN(prev => prev)` – будет ли `render`?
- 02 Чем `reducer` отличается от `functional update`?
- 03 Какие `deps` нужны `useCallback` с `functional update`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState#updating-state-based-on-the-previous-state>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

CUSTOM HOOKS**ТЕСТ****HIDDEN GLOBAL****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Дизайн собственных хуков и где не превратить их в скрытое глобальное состояние.

ВОПРОСЫ ДНЯ

- Q17 Как проектировать custom hooks?
- Q18 Что должно быть внутри custom hook, а что снаружи?
- Q19 Как тестировать custom hook?
- Q20 Как не превратить custom hook в hidden global state?

Как проектировать custom hooks?

ОПИСАНИЕ

Хороший custom hook решает одну задачу, имеет понятный API возврата (объект или кортеж), не лезет в глобальное состояние без явной причины и инкапсулирует деталь реализации. Имя обязательно начинается с use*. Внутри – только то, что относится к задаче; оставь снаружи всё, что зависит от контекста использования.

КАК РАБОТАЕТ

- 01 Один use-* хук – одна ответственность.
- 02 Возврат – кортеж as const (для пары) или объект (для трёх+).
- 03 Generic-параметры передаются вверх: useFetch<T>.
- 04 Inputs – параметры (urls, deps), outputs – state и управляющие методы.
- 05 Не использовать window, localStorage напрямую без SSR-safe-обёртки.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Переиспользуемая stateful-логика.
- 02 Wrapper'ы вокруг общего API (useStorage, useDebounceValue).
- 03 Интеграция с внешним store – useSyncExternalStore.

ПОДВОДНЫЕ КАМНИ

- 01 Хук-мегамонстр (useEverything) – рефакторить.
- 02 Хук, втягивающий context без явного провайдера – скрытая зависимость.
- 03 Хук с side effect в render – антипаттерн.
- 04 Возврат tuple без as const → потеря позиций.

КОД

// Базовый

```
function useToggle(initial = false) {  
  const [v, setV] = useState(initial);  
  const toggle = useCallback(() => setV(p => !p), []);  
  return [v, toggle] as const;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Tuple или объект для return?
- 02 Когда хук – антипаттерн?
- 03 Как избежать скрытых глобальных зависимостей?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/reusing-logic-with-custom-hooks>

Что должно быть внутри custom hook, а что снаружи?

ОПИСАНИЕ

Внутри — только логика, общая для всех использований хука: state, effect, переходы. Снаружи — всё, что специфично для конкретного потребителя: рендер, layout, обработка ошибок UI, навигация. Хук — про state-логику, не про конкретный экран.

КАК РАБОТАЕТ

- 01 Внутри хука: useState/useReducer, useEffect, useMemo/useCallback, обращения к refs.
- 02 Снаружи: JSX, Toast/Alert, navigation, аналитика конкретного UI.
- 03 Inputs: параметры → понятная зависимость хука от внешнего.
- 04 Outputs: state и actions, которые потребитель комбинирует с UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нетривиальный хук.
- 02 Хук, переиспользуемый в нескольких местах.
- 03 Хук с side effect — внутри обязательно cleanup.

ПОДВОДНЫЕ КАМНИ

- 01 Хук с router-вызовом — сложно тестировать без роутера.
- 02 Хук, рендерящий JSX — это уже компонент.
- 03 Хук, кидающий toast — связь с UI-фреймворком.

КОД

// Хорошо

```
function useUser(id: string) {  
  const [data, setData] = useState<User>();  
  useEffect(() => { fetchUser(id).then(setData); }, [id]);  
  return data;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда хук становится компонентом?
- 02 Как тестировать хук без UI?
- 03 Какие зависимости антипаттерн внутри хука?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/reusing-logic-with-custom-hooks>

ВОПРОС Q19

Как тестировать custom hook?

ОПИСАНИЕ

Стандарт – `@testing-library/react` с `renderHook`. Возвращает `result.current` – текущее значение хука после `rerender`. Можно заворачивать в `wrapper`-провайдер (тестовый Context). Async-логику ждут через `waitFor` / `act`. Изолировать побочные эффекты – `mock`-ать `fetch` или передавать инжектированные зависимости.

КАК РАБОТАЕТ

- 01 `renderHook(() => useThing())` → { `result`, `rerender`, `unmount` }.
- 02 `result.current` – текущий `return` хука.
- 03 `act(() => result.current.action())` – обернуть `state`-обновление.
- 04 Wrapper для context: `renderHook(useThing, { wrapper : MyProvider })`.
- 05 MSW для моканья `fetch` – независимо от React.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Хуки с `logic`-состоянием – обязательно тесты.
- 02 Хуки, возвращающие методы (`action`) – проверить эффект.
- 03 Хуки с `deps` – тесты для пересборки.

ПОДВОДНЫЕ КАМНИ

- 01 Без `act` – warning о неконтролируемом `state-update`.
- 02 Без `wrapper` – провайдеры недоступны.
- 03 Тест на реальном `fetch` – flaky; мок через MSW.

КОД

// Базовое

```
import { renderHook, act } from '@testing-library/react';
test('toggle', () => {
  const { result } = renderHook(() => useToggle(false));
  expect(result.current[0]).toBe(false);
  act(() => result.current[1]());
  expect(result.current[0]).toBe(true);
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Зачем `act`?
- 02 Как тестировать `async`-хук?
- 03 Чем MSW удобнее обычного `mock`?

ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/react-testing-library/api/#renderhook>

ВОПРОС Q20

Как не превратить custom hook в hidden global state?

ОПИСАНИЕ

Хук, читающий из глобального хранилища (window, localStorage, module-singleton, неподписанный shared object), без явного провайдера или зависимости – скрытое глобальное состояние. Это плохо: тесты становятся хрупкими, SSR ломается, дебаг идёт по непрозрачным путям. Лечение: провайдер контекста, useSyncExternalStore с явным store-параметром, инъекция через props.

КАК РАБОТАЕТ

- 01 Module-singleton (let store = {}) – невидимая зависимость.
- 02 Provider-pattern: useFoo() читает из ContextProvider.
- 03 useSyncExternalStore: store передаётся как параметр или из контекста.
- 04 Тесты: подменяют провайдера для разных сценариев.
- 05 SSR: каждый запрос – новый store, не общий между пользователями.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Авторизация, тема, локаль – через context-провайдеры.
- 02 Глобальный кеш – через react-query (provider-based).
- 03 Auth0/Clerk – обёртки в провайдер.

ПОДВОДНЫЕ КАМНИ

- 01 let user = null в module – два пользователя на сервере видят чужие данные.
- 02 Тест без сброса module-singleton – flaky.
- 03 Невидимый зависимости от глобалов – debugging-кошмар.

КОД

// Плохо: module-singleton

```
// shared.ts
export let user: User | null = null;
// useUser.ts
export function useUser() { return user; }
// SSR ломается, тесты – flaky
```

// Хорошо: provider

```
const UserContext = createContext<User | null>(null);
export function useUser() { return useContext(UserContext); }
// в App: <UserContext.Provider value={current}>
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие признаки скрытого global state?

- 02 Чем provider лучше module-singleton?
- 03 Как тестировать без global side effects?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/passing-data-deeply-with-context>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Назвать правила хуков и почему нельзя условно (Q1, Q2)
- ☐ Сравнить useState и useReducer (Q3, Q4, Q5)
- ☐ Объяснить useRef и чем он отличается от state (Q6, Q7)
- ☐ Сказать, когда нужны useMemo / useCallback / React.memo (Q8, Q10, Q12)
- ☐ Описать, когда мемоизация бесполезна / вредна (Q9, Q11, Q13)
- ☐ Расписать stale closure и три способа лечения (Q14, Q15, Q16)
- ☐ Сформулировать дизайн custom hook + что внутри/снаружи (Q17, Q18)
- ☐ Описать тестирование хука (Q19) и hidden global state (Q20)

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru